

GUI Refactoring Changelog

org.openmarkov.gui — Applied Changes

OpenMarkov — CISIAD, UNED

April 2026

Contents

1	Overview	3
2	R3 — Elimination of Unsafe Casts to TablePotential	3
2.1	Motivation	3
2.2	Before	3
2.3	After	4
2.3.1	Change 1: Typed getter and setter in Link.java	5
2.3.2	Change 2: Pattern matching instanceof (Java 21)	5
2.3.3	Change 3: Null safety in ProbNetCopier	6
2.4	Summary of impact	6
3	R5 — Consolidation of CPT Value-Edit Duplication	6
3.1	Motivation	6
3.2	Before	7
3.3	After	9
3.3.1	New utility method	9
3.3.2	Simplified callers	11
3.4	Design rationale	12
3.5	Summary of impact	12
4	R8 — Temporal Interval Editors (Discarded)	12
4.1	Analysis and decision	12
5	R4 — MainMenu Simplification	13
5.1	Problem	13
5.2	Before	14
5.3	After	15
5.3.1	EnumMap replaces 53 fields	15
5.3.2	Factory methods replace 46 lazy getters	15
5.3.3	Build methods replace imperative construction	16
5.3.4	Map lookup replaces 43-case switch	16
5.4	What was preserved	17
5.5	Summary of impact	17

6	R1 — MainPanelListenerAssistant Decomposition	17
6.1	Motivation	17
6.2	After	18
6.2.1	NetworkFileHandler (467 lines)	19
6.2.2	InferenceHandler (238 lines)	19
6.2.3	EditAndViewHandler (121 lines)	20
6.2.4	Facade (401 lines)	20
6.3	Design rationale	20
6.4	Summary of impact	20
7	R7 — VisualNetwork Decoupling from MainGUI	20
7.1	Motivation	21
7.2	Before	22
7.3	After	23
7.4	Summary of impact	24
8	R6 — ValuesTable Component Cleanup	24
8.1	Motivation	24
8.2	Changes	24
8.2.1	Deleted class: AugmentedValuesTableCellRenderer	24
8.2.2	Dead code in AugmentedValuesTable	24
8.2.3	Dead code in ValuesTable	25
8.2.4	Shadowed field in ICIValuesTable	25
8.2.5	Unused parameter in LinkRestrictionCellRenderer	25
8.2.6	Dead variable in LinkRestrictionValuesTable	25
8.3	Summary of impact	25
8.4	What was not changed (and why)	26
9	Verification	26

1 Overview

This document records the refactoring changes applied to the `org.openmarkov.gui` module as part of the GUI improvement plan described in `refactorizacion-gui.tex`. Each section covers one refactoring, showing the state before and after the change, the motivation, and the files affected.

Ref.	Description	Date	Status
R3	Eliminate unsafe <code>TablePotential</code> casts	April 2026	Done
R5	Consolidate CPT value-edit duplication	April 2026	Done
R8	Consolidate temporal interval editors	April 2026	Discarded

Table 1: Refactoring entries in this changelog.

2 R3 — Elimination of Unsafe Casts to `TablePotential`

2.1 Motivation

The GUI module contained 16 explicit casts to `TablePotential` and 10 `instanceof TablePotential` checks. Most of them originated from `Link.getRestrictionsPotential()`, which returned `Potential` (the abstract base type) even though the underlying field was already declared as `TablePotential`. This forced every caller to cast, hiding potential `NullPointerException` errors and cluttering the code with redundant type assertions.

2.2 Before

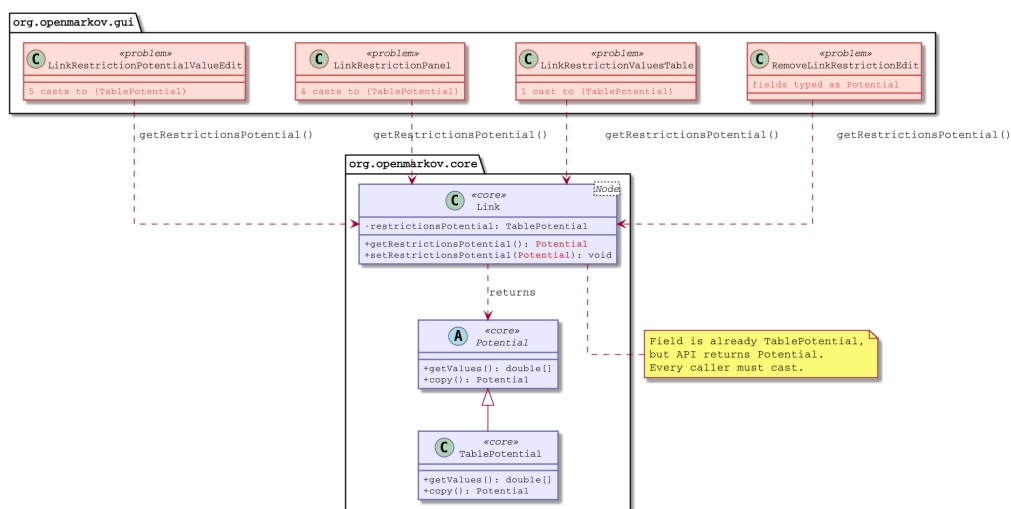


Figure 1: Before R3: `Link` exposes `Potential` in its API. Every GUI caller must cast to `TablePotential`. Diagram source: `r3-before.puml`.

The getter and setter signatures in `Link.java` were:

```
1 // Field is already TablePotential
```

```

2 private TablePotential restrictionsPotential;
3
4 // But the API returns the abstract type
5 public Potential getRestrictionsPotential() {
6     return restrictionsPotential;
7 }
8
9 public void setRestrictionsPotential(Potential potential) {
10     this.restrictionsPotential = (TablePotential) potential;
11 }

```

Listing 1: Link.java — before R3

This forced callers to write unsafe casts like:

```

1 TablePotential tp =
2     (TablePotential) link.getRestrictionsPotential();

```

Listing 2: Typical caller code — before R3

Additionally, classes like TablePotentialValueEdit used the legacy check-then-cast pattern:

```

1 this.isExactDistrPotential =
2     potential instanceof ExactDistrPotential;
3 if (getExactDistrPotential()) {
4     this.oldExactDistrPotential =
5         (ExactDistrPotential) potential;
6     this.oldTablePotential =
7         ((ExactDistrPotential) potential).getTablePotential();
8 } else {
9     this.oldTablePotential = (TablePotential) potential;
10 }

```

Listing 3: Check-then-cast pattern — before R3

2.3 After

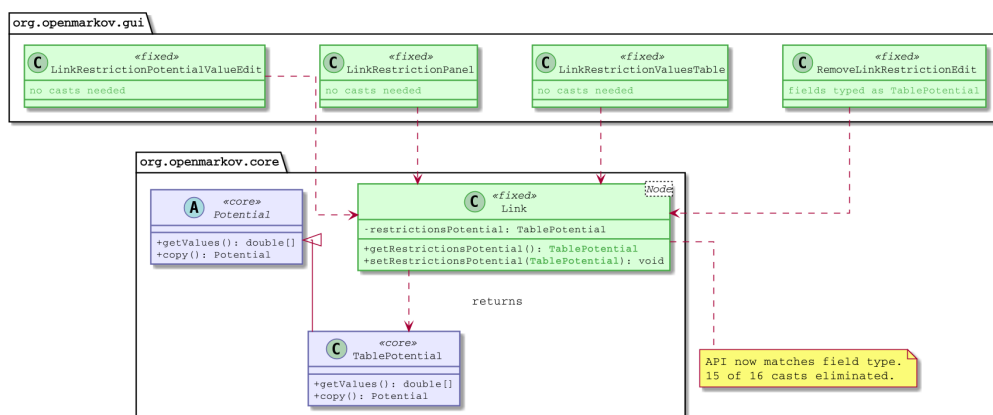


Figure 2: After R3: Link returns TablePotential directly. No casts needed in GUI callers. Diagram source: r3-after.puml.

2.3.1 Change 1: Typed getter and setter in Link.java

```

1 private TablePotential restrictionsPotential;
2
3 public TablePotential getRestrictionsPotential() {
4     return restrictionsPotential;
5 }
6
7 public void setRestrictionsPotential(TablePotential potential) {
8     this.restrictionsPotential = potential;
9 }

```

Listing 4: Link.java — after R3

This single change eliminated 10 casts across 4 GUI classes: LinkRestrictionPotentialValueEdit (5), LinkRestrictionPanel (4), LinkRestrictionValuesTable (1).

2.3.2 Change 2: Pattern matching instanceof (Java 21)

The check-then-cast pattern was replaced with Java 21 pattern matching:

```

1 if (potential instanceof ExactDistrPotential exactDistr) {
2     this.oldExactDistrPotential = exactDistr;
3     this.oldTablePotential =
4         exactDistr.getTablePotential();
5 } else if (potential instanceof TablePotential table) {
6     this.oldTablePotential = table;
7 }

```

Listing 5: Pattern matching instanceof — after R3

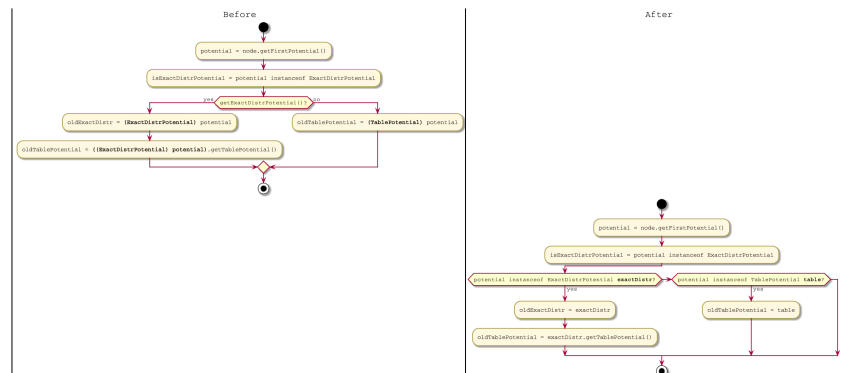


Figure 3: Flow comparison: legacy check-then-cast vs. pattern matching instanceof. Diagram source: r3-instanceof.puml.

This was applied in:

- TablePotentialValueEdit — constructor
- TablePotentialPanel — ternary expression
- ValuesTable — potential type dispatch
- PotentialsTablePanelOperations — exact distribution check

2.3.3 Change 3: Null safety in ProbNetCopier

The `getNode()` method in `ProbNet` can return `null`. Three calls in `ProbNetCopier` assumed the node would always be found (because it was cloned immediately before). We wrapped them with `Objects.requireNonNull()` to make the invariant explicit:

```
1 Objects.requireNonNull(  
2     dest.getNode(node.getName()),  
3     "Node not found in dest: " + node.getName()  
4 ).setPotentials(newPotentials);
```

Listing 6: `ProbNetCopier.java` — null safety

2.4 Summary of impact

File	Casts removed	Method
Link.java (core)	1 (internal)	Typed getter/setter
LinkRestrictionPotentialValueEdit	5	No longer needed
LinkRestrictionPanel	4	No longer needed
LinkRestrictionValuesTable	1	No longer needed
RemoveLinkRestrictionEdit	0	Fields retyped
TablePotentialValueEdit	1 of 3	Pattern matching
TablePotentialPanel	1	Pattern matching
ValuesTable	1	Pattern matching
PotentialsTablePanelOperations	1	Pattern matching
Total	15 of 16	1 cast remains

Table 2: R3 impact per file.

The single remaining cast is in `PotentialsTablePanelOperations`, where `Potential.copy()` returns `Potential` (the base type). This is inherent to the API and cannot be removed without a covariant return type on `copy()`.

3 R5 — Consolidation of CPT Value-Edit Duplication

3.1 Motivation

The probability redistribution algorithm—given an array of values and a priority list, adjust the values so they sum to 1.0—was duplicated three times across two classes:

1. `TablePotentialValueEdit` constructor (1 copy, ~40 lines)
2. `ICITablePotentialValueEdit.doEdit()` — leaky parameters (~35 lines)
3. `ICITablePotentialValueEdit.doEdit()` — noisy parameters (~35 lines, identical to leaky)

3.2 Before

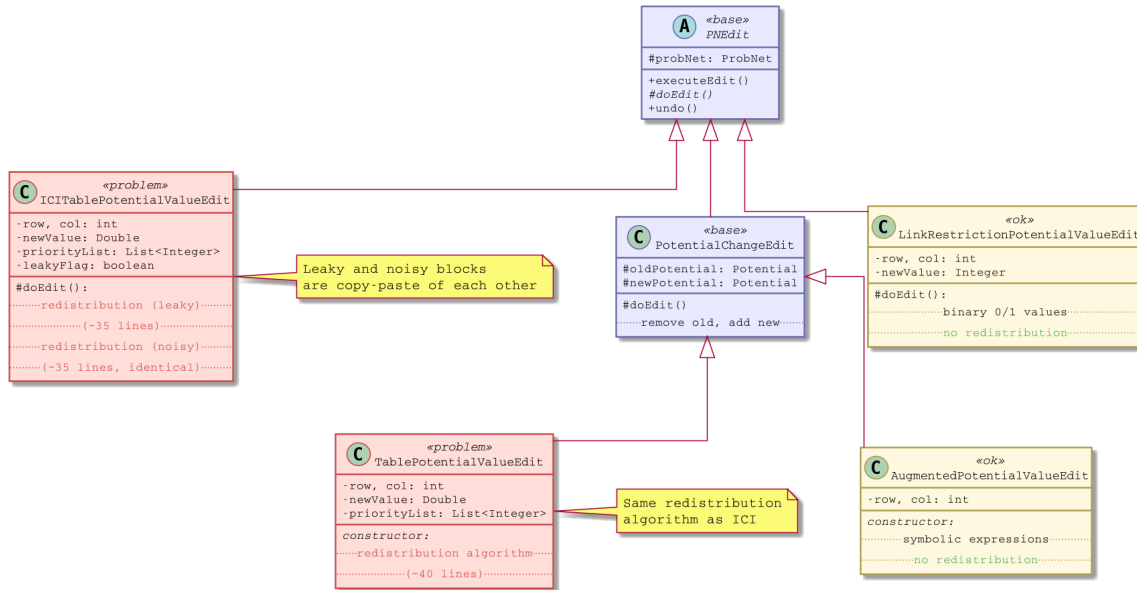


Figure 4: Before R5: the redistribution algorithm is duplicated in TablePotentialValueEdit and twice in ICITablePotentialValueEdit. Diagram source: r5-before.puml.

The redistribution in TablePotentialValueEdit (constructor):

```

1 // ~40 lines in the constructor
2 Iterator<Integer> listIterator = priorityList.listIterator();
3 double sum = 0.0;
4 int maxDecimals = 10;
5 double epsilon = Math.pow(10, -(maxDecimals + 2));
6 newTable[potentialSelected] =
7     Util.roundAndReduce(newValue, epsilon, maxDecimals);
8 while (listIterator.hasNext()) {
9     position = listIterator.next();
10    if (isEditablePosition(position)) {
11        sum = Util.roundAndReduce(
12            sum + newTable[position], epsilon, maxDecimals);
13    }
14 }
15 double rest = Math.abs(
16     Util.roundAndReduce(1 - sum, epsilon, maxDecimals));
17 if (sum > 1.0) {
18     // subtract from cells in priority order ...
19 } else {
20     // add remainder to first editable cell ...
21 }

```

Listing 7: TablePotentialValueEdit — before R5 (abbreviated)

In ICITablePotentialValueEdit.doEdit(), the same algorithm appeared twice—once for leaky parameters and once for noisy parameters—with only the array name changing:

```

1  if (leakyFlag) {
2      // ~35 lines: redistribute newLeakyParameters
3      newLeakyParameters[position] = Util.roundAndReduce(...);
4      // ... sum, rest, redistribute loop ...
5      iciPotentialEdit = new ICIPotentialEdit(
6          probNet, iciPotential, newLeakyParameters);
7  } else {
8      // ~35 lines: redistribute newNoisyParameters (identical
9          logic)
10     newNoisyParameters[position] = Util.roundAndReduce(...);
11     // ... sum, rest, redistribute loop ...
12     iciPotentialEdit = new ICIPotentialEdit(
13         probNet, iciPotential, noisyVariable, newNoisyParameters
14         );
15 }

```

Listing 8: ICITablePotentialValueEdit.doEdit() — before R5 (abbreviated)

3.3 After

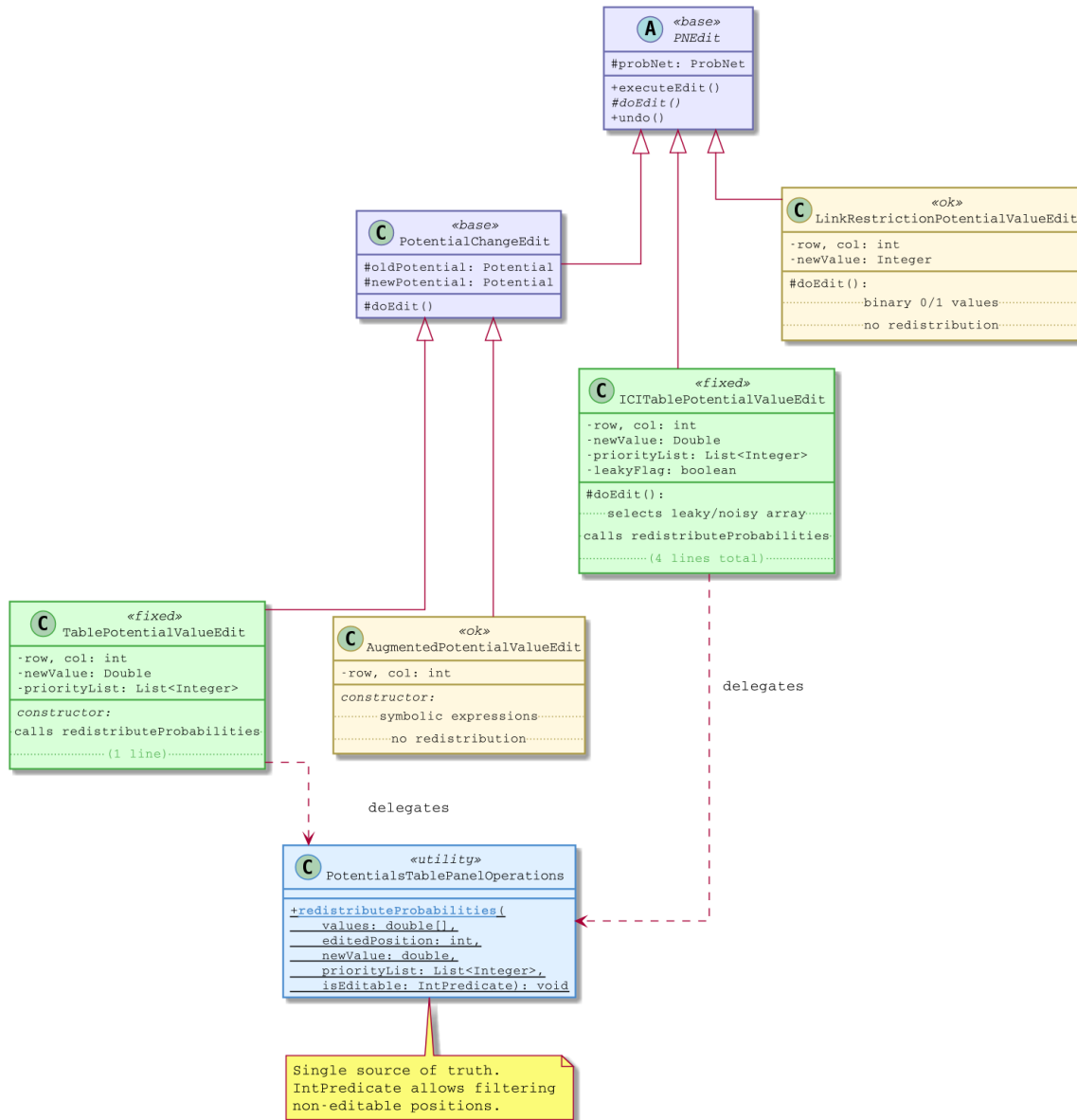


Figure 5: After R5: both classes delegate to a single `redistributeProbabilities()` method in `PotentialsTablePanelOperations`. Diagram source: `r5-after.puml`.

3.3.1 New utility method

The algorithm was extracted to a static method in `PotentialsTablePanelOperations`, which was already a utility class for table potential operations:

```

1 public static void redistributeProbabilities(
2     double[] values,
3     int editedPosition,
4     double newValue,
5     List<Integer> priorityList,
6     IntPredicate isEditable) {
7     int maxDecimals = 10;

```

```

8     double epsilon = Math.pow(10, -(maxDecimals + 2));
9     values[editedPosition] =
10         Util.roundAndReduce(newValue, epsilon, maxDecimals);
11
12     double sum = 0.0;
13     for (int pos : priorityList) {
14         if (isEditable.test(pos)) {
15             sum = Util.roundAndReduce(
16                 sum + values[pos], epsilon, maxDecimals);
17         }
18     }
19
20     double rest = Math.abs(
21         Util.roundAndReduce(1 - sum, epsilon, maxDecimals));
22     if (sum > 1.0) {
23         for (int pos : priorityList) {
24             if (rest == 0) break;
25             if (isEditable.test(pos)) {
26                 rest = Util.roundAndReduce(
27                     rest - values[pos], epsilon, maxDecimals);
28                 if (rest < 0) {
29                     values[pos] = Math.abs(
30                         Util.roundAndReduce(rest, epsilon,
31                             maxDecimals));
32                     break;
33                 }
34                 values[pos] = 0.0;
35             }
36         } else {
37             for (int pos : priorityList) {
38                 if (isEditable.test(pos)) {
39                     values[pos] = Util.roundAndReduce(
40                         values[pos] + rest, epsilon, maxDecimals);
41                     break;
42                 }
43             }
44         }
45     }

```

Listing 9: PotentialsTablePanelOperations.redistributeProbabilities()

The `IntPredicate isEditable` parameter is the key design decision: `TablePotentialValueEdit` passes a method reference `this::isEditablePosition` to filter non-editable cells, while `ICITablePotentialValueEdit` passes `pos -> true` because all ICI parameter positions are always editable.

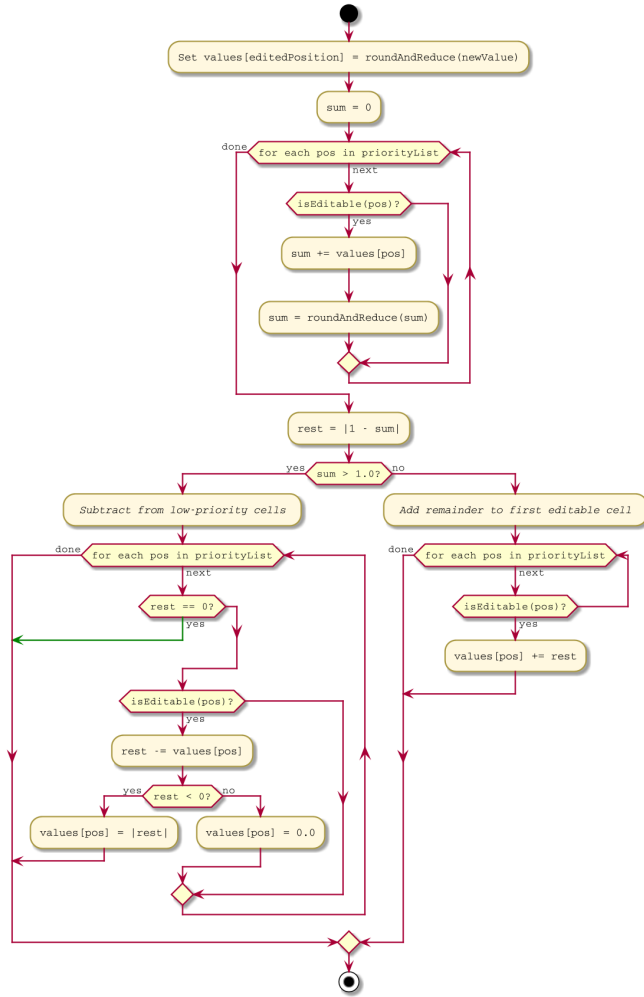


Figure 6: Flowchart of the redistribution algorithm. Diagram source: r5-algorithm.puml.

3.3.2 Simplified callers

TablePotentialValueEdit constructor (was ~40 lines):

```

1 PotentialsTablePanelOperations.redistributeProbabilities(
2     newTable, potentialSelected, newValue,
3     priorityList, this::isEditablePosition);

```

Listing 10: TablePotentialValueEdit — after R5

ICITablePotentialValueEdit.doEdit() (was ~70 lines):

```

1 double[] params = leakyFlag
2     ? newLeakyParameters : newNoisyParameters;
3 PotentialsTablePanelOperations.redistributeProbabilities(
4     params, position, newValue, priorityList, pos -> true);
5
6 ICIPotentialEdit iciPotentialEdit = leakyFlag
7     ? new ICIPotentialEdit(probNet, iciPotential,
8         newLeakyParameters)
9     : new ICIPotentialEdit(probNet, iciPotential,

```

```

10         noisyVariable , newNoisyParameters);
11     iciPotentialEdit.executeEdit();

```

Listing 11: ICITablePotentialValueEdit.doEdit() — after R5

3.4 Design rationale

The four CPT value-editing classes use two incompatible inheritance trees (`PotentialChangeEdit` vs. `PNEdit` directly), different value types (double, symbolic, binary), and different execution models (constructor vs. `doEdit()`). A common base class would have required restructuring the entire edit hierarchy for little gain.

Extracting the shared *algorithm* (redistribution) into a static utility method, parameterized with an `IntPredicate` for editability, was the simplest approach that eliminated the duplication without touching the class hierarchy.

3.5 Summary of impact

File	Lines removed	Lines added	Change
PotentialsTablePanelOperations	0	30	New utility method
TablePotentialValueEdit	40	2	Delegates to utility
ICITablePotentialValueEdit	70	6	Delegates to utility
Net	−72 lines		

Table 3: R5 impact per file.

4 R8 — Temporal Interval Editors (Discarded)

4.1 Analysis and decision

Three interval-editing classes were analyzed for possible consolidation:

Class	Lines
NodePartitionedIntervalEdit	165
RevelationIntervalEdit	152
PartitionedIntervalEdit	47
Total	364

Table 4: R8 candidate classes.

The refactoring was discarded because the three classes do not share enough structure to justify a common abstraction:

1. **Different operation models:** `PartitionedIntervalEdit` (47 lines) performs a wholesale replacement; `NodePartitionedIntervalEdit` modifies properties in place; `RevelationIntervalEdit` manages a list of intervals on a link (add, remove, modify, toggle).

2. **Incompatible action sets:** 0, 2, and 4 actions respectively—no common interface.
3. **Different backup strategies:** scalar double, object reference, or both.
4. **Low volume:** 364 lines with only partial overlap does not justify the maintenance cost of an abstraction.

5 R4 — MainMenu Simplification

5.1 Problem

MainMenu (1,529 lines) declared **53 fields** of type JMenuItem/JMenuItem/JCheckBoxMenuItem and constructed them via **46 lazy-getter methods**, each following an identical pattern:

```
1 private JMenuItem getFileNewMenuItem() {  
2     if (fileNewMenuItem == null) {  
3         fileNewMenuItem = new LocalizedMenuItem(  
4             MenuItemNames.FILE_NEW,  
5             ActionCommands.NEW_NETWORK.getCommandName());  
6         fileNewMenuItem.setIcon(IconBind.NEW_ENABLED.icon());  
7         fileNewMenuItem.setAccelerator(  
8             KeyStroke.getKeyStroke(KeyEvent.VK_N, CTRL));  
9         fileNewMenuItem.addActionListener(listener);  
10    }  
11    return fileNewMenuItem;  
12 }
```

Listing 12: Typical lazy-getter pattern (repeated 46 times)

Additionally, MainPanelMenuAssistant accessed items through a **43-case switch** in getJComponentActionCommand().

The problems were:

1. **Massive repetition:** 53 fields and 46 getters with identical structure.
2. **Maintenance cost:** adding a menu item required a field declaration, a lazy getter, adding to the parent menu, and registering in MenuItemNames.
3. **Size:** 1,529 lines made navigation difficult.
4. **Fragile switch:** the 43-case switch had to be kept in sync with the fields manually.

5.2 Before

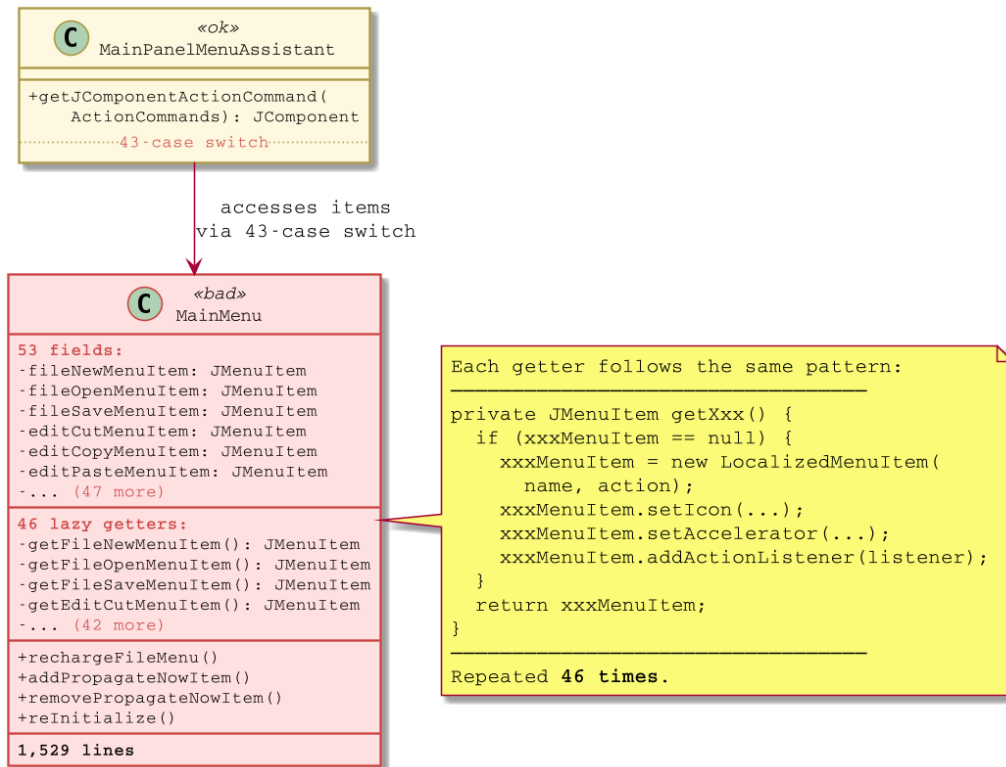


Figure 7: Before R4: 53 fields, 46 lazy getters, and a 43-case switch for item lookup. Diagram source: r4-before.puml.

5.3 After

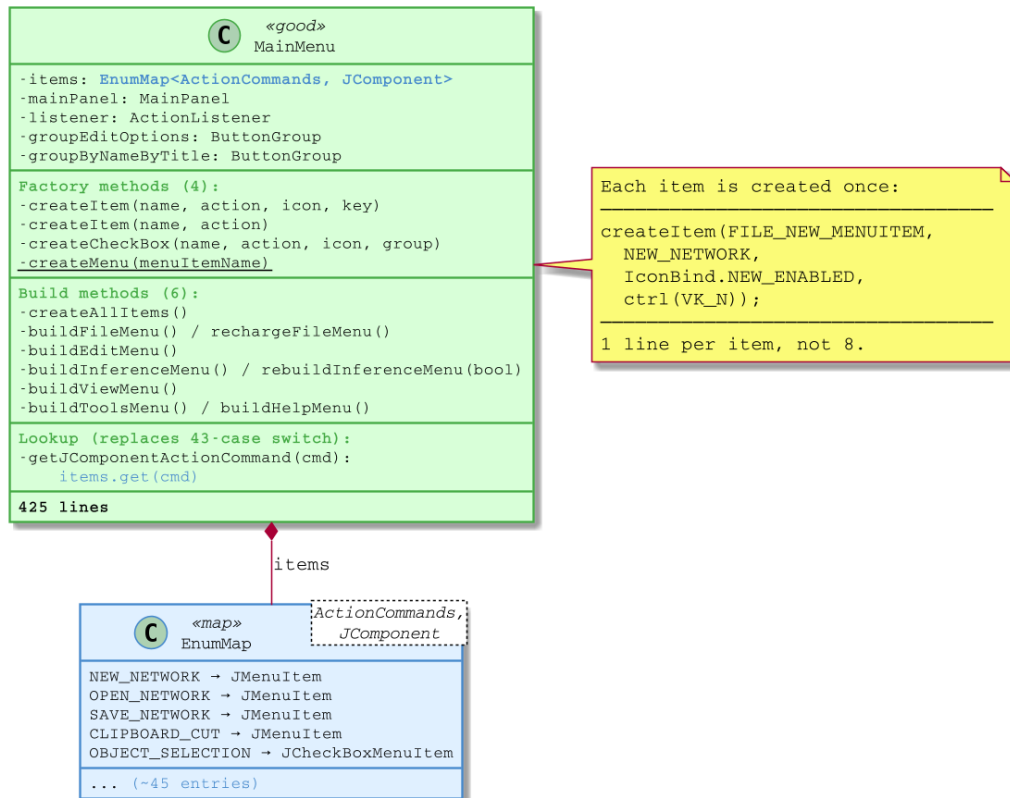


Figure 8: After R4: all items stored in an EnumMap, created via factory methods. Diagram source: r4-after.puml.

The rewrite introduced four key changes:

5.3.1 EnumMap replaces 53 fields

All menu items are stored in a single map:

```

1 private final Map<ActionCommands, JComponent> items =
2   new EnumMap<>(ActionCommands.class);
  
```

Listing 13: Central item storage

5.3.2 Factory methods replace 46 lazy getters

Two factory methods create items and register them in the map:

```

1 private void createItem(String name, ActionCommands action,
2                           IconBind icon, KeyStroke key) {
3     var item = new LocalizedMenuItem(
4         name, action.getCommandName(), icon, key);
5     item.addActionListener(listener);
6     items.put(action, item);
7 }
8
  
```

```

9 private void createCheckBox(String name, ActionCommands action,
10                             IconBind icon, ButtonGroup group) {
11     var item = icon != null
12         ? new LocalizedCheckBoxMenuItem(name, action.
13             getCommandName(), icon)
14         : new LocalizedCheckBoxMenuItem(name, action.
15             getCommandName());
16     item.addActionListener(listener);
17     if (group != null) group.add(item);
18     items.put(action, item);
19 }

```

Listing 14: Factory methods (replacing 46 lazy getters)

All items are created once in `createAllItems()`, with one line per item instead of eight:

```

1 createItem(FILE_NEW_MENUITEM, NEW_NETWORK, NEW_ENABLED, ctrl(
2     VK_N));
3 createItem(FILE_OPEN_MENUITEM, OPEN_NETWORK, OPEN_ENABLED, ctrl(
4     VK_O));
5 createItem(FILE_SAVE_MENUITEM, SAVE_NETWORK, SAVE_ENABLED, ctrl(
6     VK_S));

```

Listing 15: Declarative item creation (excerpt)

5.3.3 Build methods replace imperative construction

Menu population reads like a declarative spec:

```

1 private JMenu buildEditMenu() {
2     editMenu = createMenu(MenuItemNames.EDIT_MENU);
3     editMenu.add(items.get(ActionCommands.CLIPBOARD_CUT));
4     editMenu.add(items.get(ActionCommands.CLIPBOARD_COPY));
5     editMenu.add(items.get(ActionCommands.CLIPBOARD_PASTE));
6     editMenu.add(items.get(ActionCommands.OBJECT_REMOVAL));
7     editMenu.addSeparator();
8     editMenu.add(items.get(ActionCommands.UNDO));
9     editMenu.add(items.get(ActionCommands.REDO));
10    // ...
11    return editMenu;
12 }

```

Listing 16: Menu builder (example: Edit menu)

5.3.4 Map lookup replaces 43-case switch

```

1 // BEFORE: 43-case switch (~50 lines)
2 switch (actionCommand) {
3     case "NewNetwork": return fileNewMenuItem;
4     case "OpenNetwork": return fileOpenMenuItem;
5     case "SaveNetwork": return fileSaveMenuItem;
6     // ... 40 more cases
7 }

```



```

8
9 // AFTER: map lookup (3 lines)
10 private JComponent getJComponentActionCommand(String
    actionCommand) {
11     ActionCommands cmd = ActionCommands.of(actionCommand);
12     if (cmd == null) return null;
13     return items.get(cmd);
14 }

```

Listing 17: Item lookup — before vs. after

5.4 What was preserved

- **View menu inline lambdas:** tab navigation (Go Next/Previous Tab) and scale dialog use complex logic tied to `MainPanel` — not suitable for the factory.
- **Recent files:** dynamically loaded from `LocalPreferences` on each `rechargeFileMenu()` call.
- **Tools menu plugins:** loaded by `ToolPluginManager` discovery at runtime.
- **Inference dual configuration:** `addPropagateNowItem()` / `removePropagateNowItem()` simplified to a single `rebuildInferenceMenu(boolean)` that adds or omits the propagate item.
- **Full public API:** all methods called from other classes (`rechargeFileMenu`, `reInitialize`, `setOptionEnabled`, etc.) preserved with identical signatures.

5.5 Summary of impact

Metric	Before	After	Change
Total lines	1,529	425	−1,104 (−72%)
Fields	53	1 map	−52 declarations
Lazy getters	46	0	Replaced by 2 factories
Switch cases	43	0	Replaced by map lookup

Table 5: R4 impact summary.

6 R1 — MainPanelListenerAssistant Decomposition

6.1 Motivation

`MainPanelListenerAssistant` (1,534 lines) was the central GUI controller — a God Object that handled *all* application events through a single `actionPerformed()` method with **67 switch cases**. It mixed file I/O, inference mode management, edit operations, view operations, dialog launchers, and decision analysis in one class.



Figure 9: Before R1: monolithic controller with 1,534 lines mixing five responsibilities. Diagram source: `r1-before.puml`.

Additionally, the class contained ~100 lines of commented-out dead code.

6.2 After

The class was decomposed into a thin facade plus three domain-specific handlers:

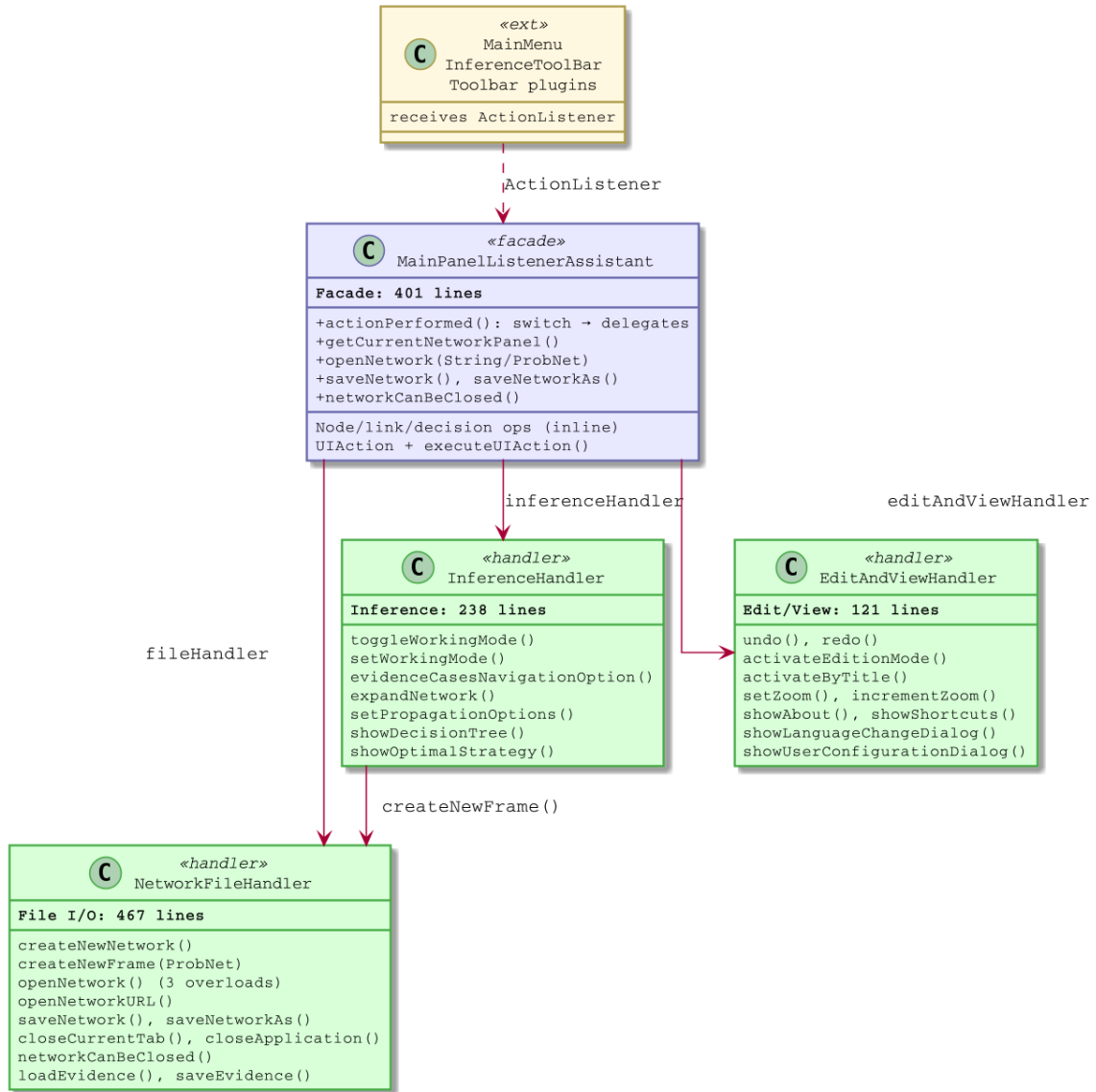


Figure 10: After R1: facade delegates to three domain handlers. Diagram source: `r1-after.puml`.

6.2.1 NetworkFileHandler (467 lines)

All file I/O operations: open (3 overloads), save, save-as, close, backup, evidence load-/save, network creation, and file chooser dialogs. This handler owns mutations to the shared `networkPanels` list.

6.2.2 InferenceHandler (238 lines)

Inference mode switching, evidence case navigation, propagation and inference options, network expansion, decision tree, and optimal strategy display. Uses `NetworkFileHandler.createNewFrame()` for the expand-network operation.

6.2.3 EditAndViewHandler (121 lines)

Undo/redo, edition mode activation, zoom, by-title/by-name toggle, and dialog launchers (configuration, shortcuts, about, language change).

6.2.4 Facade (401 lines)

MainPanelListenerAssistant retains:

- `actionPerformed()` — the switch statement stays, but each case is a one-liner delegation to a handler.
- `UIAction / executeUIAction()` — the shared exception-wrapping pattern.
- Public API methods as thin delegates (`getCurrentNetworkPanel`, `openNetwork`, `saveNetwork`, etc.).
- Inline node/link/decision operations (~80 lines) — too scattered to justify a fourth handler.
- `WindowListener` and `ComponentListener` methods.

6.3 Design rationale

The handlers are **package-private** classes in `org.openmarkov.gui.window`. No new public API is introduced. External callers (9 modules, 35+ references to `getCurrentNetworkPanel()`) are completely unaffected because the facade preserves all public method signatures.

The `ActionListener` interface stays on the facade, so `MainMenu`, `InferenceToolBar`, and toolbar plugins (discovered via reflection in `ToolBarManager`) continue to receive the same `ActionListener` instance.

6.4 Summary of impact

File	Lines	Role
MainPanelListenerAssistant (facade)	401	Event dispatch + public API
NetworkFileHandler	467	File I/O, open/save/close
InferenceHandler	238	Inference mode, evidence
EditAndViewHandler	121	Edit, view, dialogs
Total	1,227	
Original	1,534	
Net reduction	-307 (-20%)	Dead code removed

Table 6: R1 impact: file sizes after decomposition.

The primary benefit is not line reduction but **separation of concerns**: each handler has a single responsibility and can be understood, tested, and modified independently.

7 R7 — VisualNetwork Decoupling from MainGUI

Files changed: `graphic/VisualNetwork.java`, `window/edition/NetworkPanel.java`

7.1 Motivation

`VisualNetwork` is a view-model class in the `graphic/` layer. It implements `PNEditListener` to receive notifications when edits are executed, undone, or redone on a `ProbNet`, and rebuilds the visual representation accordingly.

However, it also held a `MainGUI` reference and used it to directly enable/disable undo and redo toolbar buttons:

```
1 mainGUI.mainPanel.getEditionToolBar()  
2   .getUndoButton().setEnabled(canUndo);  
3 mainGUI.mainPanel.getEditionToolBar()  
4   .getRedoButton().setEnabled(canRedo);
```

This created **upward coupling**: a view-model class reached into the UI layer (`MainGUI` → `MainPanel` → `EditionToolBar`) to toggle button state. This violates the layered architecture and makes `VisualNetwork` harder to test and reuse.

The same undo/redo button updates are already performed by `MainPanelMenuAssistant`, which is also a `PNEditListener` on the same `PNESupport`. Its `updateUndoRedo()` method calls `setOptionEnabled(ActionCommands.UNDO, canUndo)` and `setOptionEnabled(ActionCommands.canRedo)`. The `setOptionEnabled()` method, inherited from `MenuAssistant`, iterates over all registered `MenuToolBarBasic` implementations — which include both `MainMenu` and the toolbar classes. Therefore, the toolbar button updates in `VisualNetwork` were fully redundant.

7.2 Before

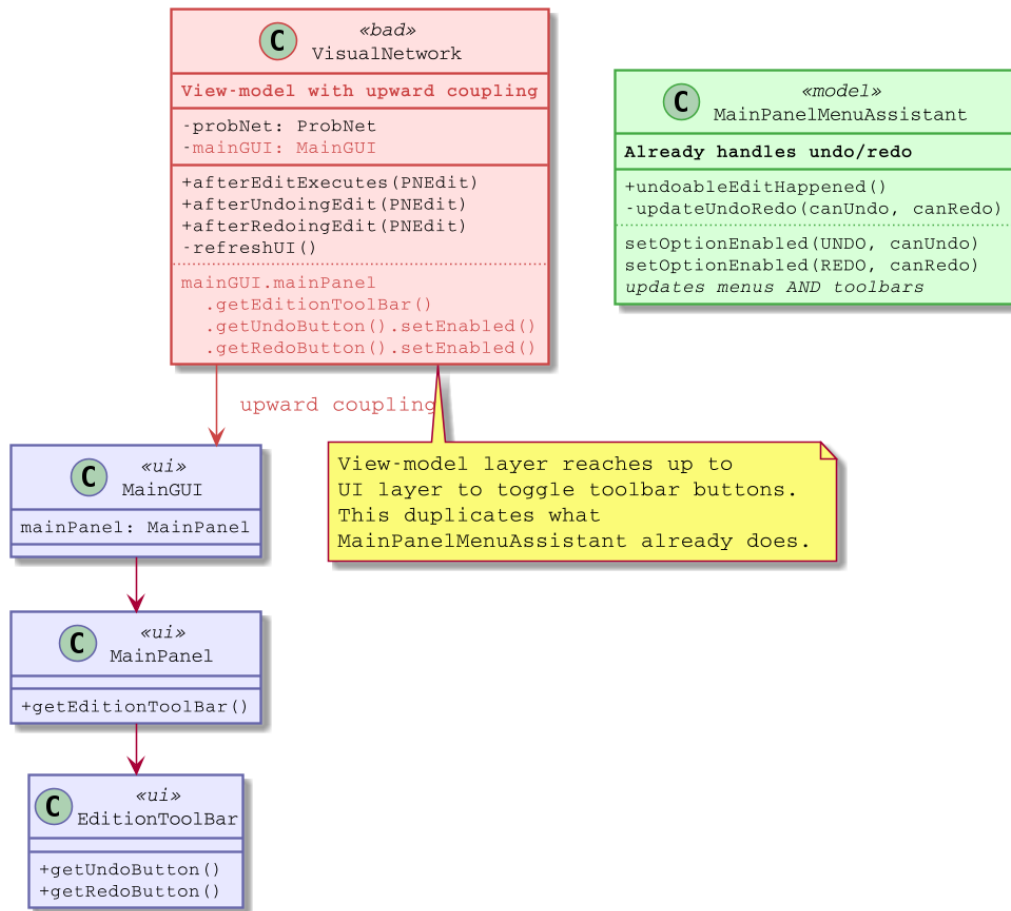


Figure 11: VisualNetwork coupled to MainGUI for undo/redo toolbar updates.

The MainGUI field was used in exactly 4 lines across two methods:

- afterEditExecutes(PNEdit) — lines 935–936
- refreshUI() (called by afterUndoingEdit and afterRedoingEdit) — lines 976–978

7.3 After

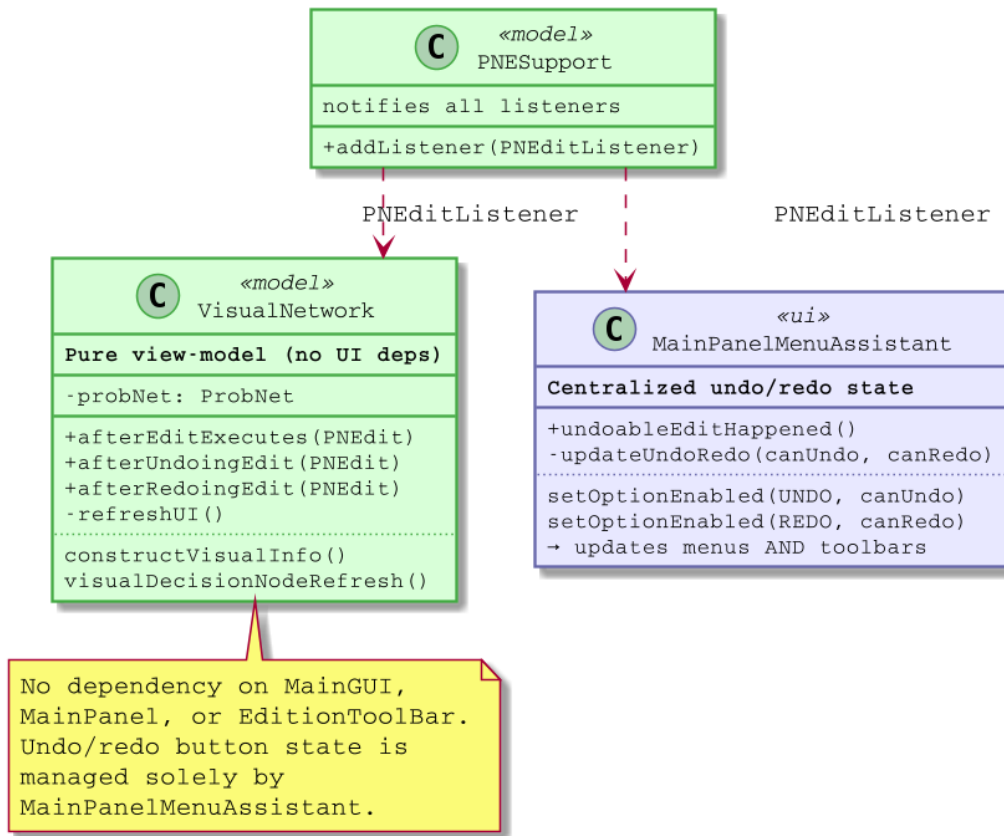


Figure 12: VisualNetwork decoupled: undo/redo state managed solely by MainPanelMenuAssistant.

Changes applied:

1. Removed the mainGUI field from VisualNetwork.
2. Removed the MainGUI parameter from the constructor (signature changed from VisualNetwork(ProbNet, MainGUI) to VisualNetwork(ProbNet)).
3. Removed the 4 lines of direct toolbar button manipulation from afterEditExecutes() and refreshUI().
4. Removed the MainGUI import.
5. Updated the single call site in NetworkPanel.java to use the new one-argument constructor.

7.4 Summary of impact

File	Before	After	Δ	Change
VisualNetwork.java	1,172	1,161	-11	Remove <code>mainGUI</code> field, constructor param, toolbar updates
NetworkPanel.java	—	—	-1	Remove <code>mainGUI</code> argument from constructor call

Table 7: R7 impact.

The line reduction is small, but the architectural benefit is significant: `VisualNetwork` no longer depends on the UI layer and respects the view-model boundary. The undo/redo button state is managed by a single mechanism (`MainPanelMenuAssistant.setOptionEnabled()`), eliminating redundancy and the risk of inconsistent updates.

8 R6 — ValuesTable Component Cleanup

Files changed: 6 files in `component/` and `dialog/link/`. One class deleted.

8.1 Motivation

The `component/` package contains a hierarchy of 10 classes (4 table classes, 3 models, 6 renderers; 1,919 lines total) for editing conditional probability tables (CPTs). After analysis, the subclass overrides of `setValueAt()` and `afterEditExecutes()` represent legitimate template method usage rather than fragile inheritance, and the renderers implement genuinely different coloring logic. A full consolidation would carry high regression risk for modest structural benefit.

Instead, this refactoring focused on removing dead code, fixing a shadowed field, and eliminating unused parameters — concrete improvements that reduce noise without touching the editing or rendering logic.

8.2 Changes

8.2.1 Deleted class: `AugmentedValuesTableCellRenderer`

This renderer (32 lines) was never instantiated anywhere. `AugmentedValuesTable` inherits its renderer setup from `ValuesTable.defaultConfiguration()`, which sets `ValuesTableCellRenderer` directly. The file was deleted.

8.2.2 Dead code in `AugmentedValuesTable`

Three methods removed:

- `printTable()` (20 lines): Debug method overriding the parent’s identically-named dead method. Neither version was ever called.

- `editCellAt()` override (8 lines): Called `super.editCellAt()` then added `System.err.println("COUNT...")`. Leftover debug code. Removing it lets the parent's version (which includes `selectAll()` for keyboard/mouse cell entry) apply correctly.
- `selectAll(EventObject)` (21 lines): Private method, never called. The parent's `editCellAt()` invokes the parent's own private `selectAll()` (Java resolves private methods at compile time). This copy was unreachable.

Six imports made unnecessary by these removals were also deleted.

8.2.3 Dead code in `ValuesTable`

`printTable()` (20 lines) removed — debug method never called from any code in the project.

8.2.4 Shadowed field in `ICIValuesTable`

`ICIValuesTable` declared `private int lastCol = -1`, which shadows the parent's `protected int lastCol = -1` (`ValuesTable.java`, line 152). Since the parent field is `protected`, the subclass can access it directly. The shadow was removed.

8.2.5 Unused parameter in `LinkRestrictionCellRenderer`

The constructor accepted a `TablePotential potential` parameter that was never used. The parameter was removed from the signature, and the 3 call sites in `LinkRestrictionPanel.setCellR` were updated. The now-unnecessary `TablePotential` import was also removed.

8.2.6 Dead variable in `LinkRestrictionValuesTable`

The constructor assigned `ProbNet net = node1.getProbNet()` but never used `net`. The dead variable (and its source variable `node1`) were removed, along with the unused `ProbNet` import.

8.3 Summary of impact

File	Before	After	Δ	Change
<code>AugmentedValuesTableCellRenderer</code>	32	—	−32	Deleted (dead class)
<code>AugmentedValuesTable.java</code>	267	183	−84	Dead methods + imports
<code>ValuesTable.java</code>	873	852	−21	Dead <code>printTable()</code>
<code>ICIValuesTable.java</code>	142	138	−4	Shadowed field
<code>LinkRestrictionCellRenderer.java</code>	38	36	−2	Unused param
<code>LinkRestrictionValuesTable.java</code>	109	100	−9	Dead variable + import
Total	1,461	1,309	−152	

Table 8: R6 impact: dead code removal and cleanup.

8.4 What was not changed (and why)

- **Renderer consolidation:** The 5 remaining renderers implement genuinely different `setCellColors()` logic (ICI header coloring by parent variable, optimal policy max-detection, link restriction incompatibility coloring). Merging them into a single class would require a strategy pattern that adds complexity rather than reducing it.
- **PNEditListener extraction:** The listener callbacks (`afterEditExecutes`, `afterUndoingEdit`) are tightly coupled to the table's data model — they call `getModel().setValueAt()` with specific row/column positions derived from edit objects. Separating them would require the controller to have intimate knowledge of the table's column and row structure.
- **Replacing inheritance with configuration:** Each subclass overrides `setValueAt()` and `afterEditExecutes()` with fundamentally different edit types (`TablePotentialValueEdit`, `ICITablePotentialValueEdit`, `AugmentedPotentialValueEdit`, `LinkRestrictionPotentialValueEdit`). This is legitimate template method usage, not fragile inheritance.

9 Verification

All changes (R3, R5, R8, R4, R1, R7, R6) were verified by:

1. Full compilation of all modules: `mvn install -DskipTests` — **BUILD SUCCESS**
2. GUI module tests: `mvn test -pl org.openmarkov.gui` — **71 tests, 0 failures, 0 errors**
3. Core module tests: `mvn test -pl org.openmarkov.core` — **all passed**